Sumario

Colección de apuntes de Base de datos y MySQL....4
Introducción a las Bases de Datos....4
Motores de bases de datos....4
Repositorio de datos....5
Enfoque de una base de datos....5
Control de concurrencias....6
Conceptos Preliminares....7
Tablas....7
Tipos de datos....7
Datos numéricos....7
Estructura del Lenguaje SQL....9
DDL....9
DML....10
DCL....10
TCL....10
ANSI SQL....10
Creación de una Base de Datos....11
Creación de una Tabla....11
Eliminar una Tabla....12
Restricciones de las Tablas....12
Restricciones en las columnas....12
Otros Comandos MySQL....13
Insertar registros en la tabla....14
Selección de registros....15
Sentencia Select....15
Condiciones Lógicas....16
Tabla de Operadores Lógicos....16
Cláusulas Especiales....16
Sentencia BETWEEN....16
Sentencia IN....17
Sentencia LIKE....17
Otros ejemplos de select....18
Consultas avanzadas....25
Comando DML Insert....27
Forma básica....27
Ejemplo....27
Formas avanzadas....27
Copia de filas de otras tablas....28
Comando DML Delete....29
Ejemplo de aplicación:....29
Comando DML Update....29
Actualizaciones selectivas....29
Funciones de Agrupamiento....30
Función COUNT()....30
Función SUM()....30
Funciones MIN() y MAX()....30

Función AVG()....31
Agrupación con GROUP BY....31
Agrupación con HAVING....31
Consultas de más de una tabla....32
Uso del Join....33
Modelo de Entidad - Relación....33
Introducción....33
Entidad....33
Clave Foránea (FOREIGN KEY)....33
Conceptos Claves....34
Super llave....34
Clave candidata....34
Relación....35
Cardinalidad de las Relaciones....35
Atributos....35
Normalización de Base de Datos....35
Dependencias....36
Dependencia funcional....36
Propiedades de la dependencia funcional....36
Dependencia funcional reflexiva....36
Dependencia funcional argumentativa....36
Dependencia funcional transitiva....36
Propiedades deducidas....37
Unión....37
Pseudo-Transitiva....37
Descomposición....37
Formas normales....37
Primera Forma Normal (1FN)....38
Segunda Forma Normal (2FN)....38
Tercera Forma Normal (3FN)....39
Forma normal de Boyce-Codd (FNBC)....39
Cuarta Forma Normal (4FN)....39
Quinta Forma Normal (5FN)....40
Reglas de Codd....40
Regla 1: La regla de la información....40
Regla 2: La regla del acceso garantizado....40
Regla 3: Tratamiento sistemático de los valores nulos....41
Regla 4: La regla de la descripción de la base de datos....41
Regla 5: La regla del sub-lenguaje integral....41
Regla 6: La regla de la actualización de vistas....42
Regla 7: La regla de insertar y actualizar....42
Regla 8: La regla de independencia física....42
Regla 9: La regla de independencia lógica....42
Regla 10: La regla de la independencia de la integridad....42
Las reglas de integridad....42
Regla 11: La regla de la distribución....43
Regla 12: Regla de la no-subversión....43

Colección de apuntes de Base de datos y MySQL

No poseo los derechos de autor de todo el material expuesto en esta colección de apuntes. Utilice innumerables fuentes de datos. Algunas partes las escribí o las modifique para adaptarlas a las ultimas versiones MySQL.

Muchas gracias a todas las fuentes de datos que enriquecieron este material de estudio no comercial.

Introducción a las Bases de Datos

Una base de datos, o banco de datos, es un conjunto de datos pertenecientes a un mismo contexto y almacenados sistemáticamente para su posterior uso. En este sentido, una biblioteca puede considerarse una base de datos compuesta en su mayoría por documentos y textos impresos en papel e indexados (ordenados) para su consulta.
Actualmente, y debido al desarrollo tecnológico de campos como la informática y la electrónica, la mayoría de las bases de datos están en formato digital (electrónico), que ofrece un amplio rango de soluciones al problema de almacenar datos.

Motores de bases de datos

Un motor de Bases de Datos es un conjunto de programas que permiten el almacenamiento, modificación y extracción de la información en una base de datos, además de proporcionar herramientas para añadir, borrar, modificar y analizar los datos.

Los usuarios pueden acceder a la información usando herramientas específicas de consulta y/o de generación de informes, o bien mediante aplicaciones al efecto.

En base a esto, podemos definir que los motores de base de datos sirven para definir, construir y manipular una base de datos.

1- Definir una base de datos: consiste en especificar los tipos de datos, estructuras y restricciones para los datos que se almacenarán.

2- Construir una base de datos: es el proceso de almacenar los datos sobre algún medio de almacenamiento.

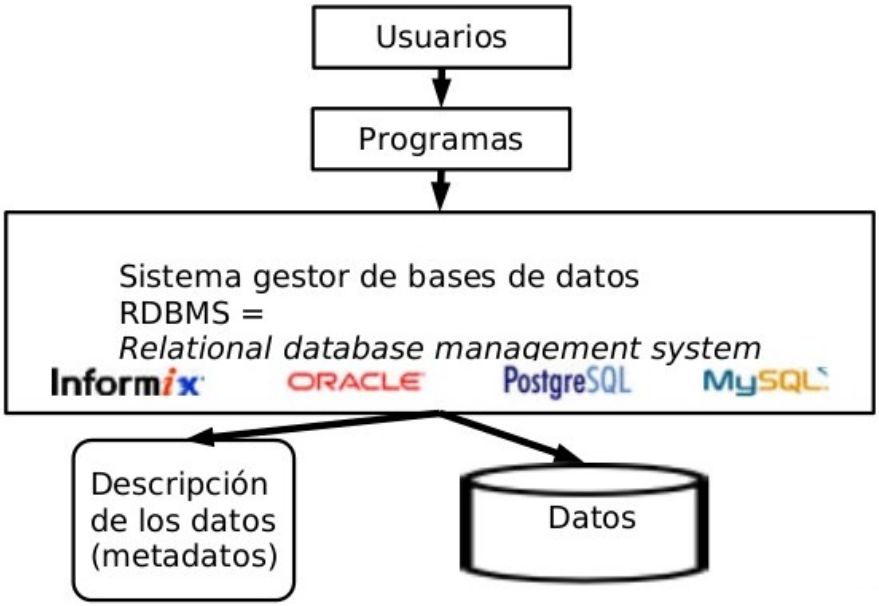
3- Manipular una base de datos: incluye funciones como consulta, actualización, etc. de bases de datos.

Las principales marcas del mercado son **ORACLE**, **MySQL**, **SQL Server**, **PostGreSQL**, **SQLite.**

Repositorio de datos

1- Un repositorio, depósito, o archivo de datos es un sitio centralizado donde se almacena y mantiene información digital, habitualmente de bases de datos o archivos informáticos.
2- Los depósitos de datos pueden distribuirse habitualmente sirviéndose de una red informática como Internet o en un medio físico. Pueden ser de acceso público o pueden estar protegidos y necesitar de una autentificación previa. Los depósitos más conocidos son los de carácter académico e institucional.
3- A diferencia de los ordenadores personales o de escritorio, los depósitos suelen contar con sistemas de respaldo (Backup) para mantenimiento preventivo y correctivo, lo que hace que la información se pueda recuperar en el caso que la máquina quede inutilizable.
4- Los depósitos se utilizan de forma intensiva en sistemas GNU/Linux, almacenando en su mayoría paquetes de software disponibles para su instalación mediante un gestor de paquetes.

Enfoque de una base de datos



1- **Dato:** Unidad mínima de información, sin sentido en sí misma, pero que adquiere significado en conjunto con otras precedentes de la aplicación que las creó. Es un conjunto de símbolos que unidos de cierta forma dan un significado lógico.
2- **Definición de Datos:** realiza una descripción de la estructura de los datos (su tipología, la forma en que se relacionan, etc.), de las operaciones que pueden realizarse con ellos (añadir, eliminar, modificar, recuperar) y de las restricciones referentes a su integridad (aquellas condiciones que todos los datos deben respetar para que se transmita la información de manera correcta).
3- **Motores de bases de datos relacionales:** son un tipo de software muy específico, dedicado a servir de interfaz entre la base de datos, el usuario y las aplicaciones que la utilizan. El propósito general de los motores de bases de datos es el de gestionar de manera clara, sencilla y ordenada un conjunto de datos que posteriormente se convertirán en información relevante para una organización.
4- **Aplicaciones:** usan el motor de datos para acceder a la información y luego poder presentarla al usuario. Las aplicaciones son desarrolladas por desarrolladores (programadores) y pueden ser escritas en diversos lenguajes. Los lenguajes más comunes son
JAVA, PHP, Visual Basic, C#, C++ , etc.
5- **Usuarios:** Son las personas que pueden acceder a los datos. Ellos pueden ser catalogados como Usuarios Comunes, Usuarios Avanzados, Desarrolladores, Administradores.

Control de concurrencias

Debido a que los sistemas de bases de datos brindan servicios en forma simultánea a una comunidad de usuarios y desarrolladores,
que normalmente están por fuera del ordenador que posee la base de datos (conectados vía red local, Internet, Intranet, etc.), se hace
imprescindible contar con un control de concurrencia que revise la consistencia de cada operación y asegura que no se creen
conflictos por usar en forma de escritura un mismo registro en forma simultánea.

Conceptos Preliminares

Tablas

1- Las tablas son objetos compuestos por una estructura (conjunto de columnas) que almacenan información interrelacionada (filas) acerca de algún objeto en general.
2- Las tablas tienen un solo nombre y es único en toda la base datos.
3- Están compuestas por registros (filas) y campos (columnas).
4- Los registros y campos pueden estar en diferentes órdenes.
5- Una base de datos puede contener varias tablas, y cada tabla almacena información con respecto a un objeto en particular.

Libros

ISBN	Título	Nombre Autor	Apellido Autor	Precio
978-0062511409	El Alquimista	Paulo	Coelho	$8.46
978-0307744593	Aleph	Paulo	Coelho	$12.23
978-034580704	El peregrino	Paulo	Coelho	$12.20

Ejemplo de tabla de 5 campos (columnas) y 3 registros

Tipos de datos

Al diseñar nuestras tablas tenemos que especificar el tipo de datos y tamaño que podrá almacenar cada campo. Una correcta elección debe procurar que la tabla no se quede corta en su capacidad, que destine un tamaño apropiado a la longitud de los datos, y la máxima velocidad de ejecución Básicamente MySQL admite dos tipos de datos: números y cadenas de caracteres. Junto a estos dos grandes grupos, se admiten otros tipos de datos especiales: formatos de fecha, etc.

Datos numéricos

En este tipo de campos sólo pueden almacenarse números, positivos o negativos, enteros o decimales, en notación hexadecimal, científica o decimal.
Los tipos numéricos tipo integer admiten los atributos SIGNED y UNSIGNED indicando en el primer caso que pueden tener valor negativo, y solo positivo en el segundo.
Los tipos numéricos pueden además usar el atributo ZEROFILL en cuyo caso los números se completarán hasta la máxima anchura disponible con ceros (columna INT(5) zerofill => valor 23 se almacenará como 00023).

BIT o BOOL: Para un número entero que puede ser 0 ó 1.
TINYINT: Es un número entero con rango de valores válidos desde -128 a 127. Si se configura como unsigned (sin signo), el rango de valores es de 0 a 255.

SMALLINT: Para números enteros, con rango desde -32768 a 32767. Si se configura como unsigned, 0 a 65535.
MEDIUMINT: Para números enteros. El rango de valores va desde -8.388608 a 8388607. Si se configura como unsigned, 0 a 16777215.
INT: Para almacenar números enteros, en un rango de -2147463848 a 2147483647. Si configuramos este dato como unsigned, el rango es 0 a 4294967295.
BIGINT: Número entero con rango de valores desde -9223372036854775808 a 9223372036854775807. Unsigned, desde 0 a 18446744073709551615.
FLOAT:Representa números decimales.
DOUBLE: Número de coma flotante de precisión doble. Es un tipo de datos igual al anterior cuya única diferencia es el rango numérico que abarca.
DECIMAL(m,d): Almacena los números como cadenas. Podemos especificar cuántos dígitos (m) pueden utilizarse (término también conocido como ancho de pantalla), y cuantos en la parte decimal (d). MySQL redondeará el decimal para ajustarse a la capacidad.

Caracteres o cadenas de texto
Las Cadenas de texto se utilizan para almacenar una serie de caracteres, palabras y/o frases de texto en donde cada carácter es lo mismo que un byte.
Este tipo se utiliza para almacenar cadenas de longitud fija. Su longitud abarca desde 1 a 255 caracteres.
CHAR: Un campo CHAR ocupará siempre el máximo de longitud que le hayamos asignado, aunque el tamaño del dato sea menor (añadiendo espacios adicionales que sean precisos).
Al igual que el anterior se utiliza para almacenar cadenas, en el mismo
VARCHAR: rango de 1-255 caracteres, pero en este caso, de longitud variable. VARCHAR solo almacena la longitud del dato, permitiendo que el tamaño de la base de datos sea menor(*)
TINYTEXT: Texto de longitud variable que puede tener hasta 255 caracteres.
TEXT: Texto de longitud variable que puede tener hasta 65535 caracteres.
MEDIUMTEXT: Texto de longitud variable que puede tener hasta 16777215 caracteres.
LONGTEXT: Texto de longitud variable que puede tener hasta 4294967295 caracteres.
Tipo de datos Binarios:
Dato binario que puede almacenar archivos o texto. En este caso, los tipos **TINYBLOB, BLOB, MEDIUMBLOB y LONGBLOB** son idénticos a sus **BLOB** homólogos TEXT, con la diferencia de que las búsquedas en un tipo BLOB tienen en cuenta las mayúsculas y minúsculas
(*) El acceso a los datos CHAR es más rápido que VARCHAR. No pueden alternarse columnas CHAR y VARCHAR en la misma tabla. MySQL cambiará las columnas CHAR a VARCHAR. También cambia automáticamente a CHAR si usamos VARCHAR con valor de 4 o menos.

Otros Tipos de Datos
DATE: Para almacenar fechas. El formato por defecto es YYYY MM DD desde 0000 00 00 a 9999 12 31
DATETIME: Combinación de fecha y hora. El rango de valores va desde el 1 de enero del 1001 a las 0 horas, 0 minutos y 0 segundos al 31 de diciembre del 9999 a las 23 horas, 59 minutos y 59 segundos. El formato de almacenamiento es de año-mes-día horas:minutos:segundos
TIME: Almacena una hora. El rango de horas va desde -838 horas, 59 minutos y 59 segundos a 838, 59 minutos y 59 segundos. El formato de almacenamiento es de 'HH:MM:SS'
YEAR: Almacena un año. El rango de valores permitidos va desde el año 1901 al año 2155. El campo puede tener tamaño dos o tamaño 4 dependiendo de si queremos almacenar el año con dos o cuatro dígitos.

Nota: Existen otros tipos de datos que no estamos utilizando en este curso

Estructura del Lenguaje SQL

SQL (por sus siglas en inglés **Structured Query Language**; en español **lenguaje de consulta estructurada**) es un lenguaje específico del dominio utilizado en programación, diseñado para administrar, y recuperar información de sistemas de gestión de bases de datos relacionales1. Una de sus principales características es el manejo del álgebra y el cálculo relacional para efectuar consultas con el fin de recuperar, de forma sencilla, información de bases de datos, así como realizar cambios en ellas.

Originalmente basado en el álgebra relacional y en el cálculo relacional, SQL consiste en un lenguaje de definición de datos, un lenguaje de manipulación de datos y un lenguaje de control de datos. El alcance de SQL incluye la inserción de datos, consultas, actualizaciones y borrado, la creación y modificación de esquemas y el control de acceso a los datos. También el SQL a veces se describe como un lenguaje declarativo, también incluye elementos procesales.

SQL fue uno de los primeros lenguajes comerciales para el modelo relacional de Edgar Frank Codd como se describió en su papel de 1970 *El modelo relacional de datos para grandes bancos de datos compartidos*. A pesar de no adherirse totalmente al modelo relacional descrito por Codd, pasó a ser el lenguaje de base de datos más usado.

SQL pasó a ser el estándar del Instituto Nacional Estadounidense de Estándares (ANSI) en 1986 y de la Organización Internacional de Normalización (ISO) en 1987. Desde entonces, el estándar ha sido revisado para incluir más características. A pesar de la existencia de ambos estándares, la mayoría de los códigos SQL no son completamente portables entre sistemas de bases de datos diferentes sin ajustes.

Existen tres clasificaciones del lenguajes para el manejo de bases de datos según el propósito: DDL, DML, DCL y TCL.

DDL

DDL → Data Definition Language: Lenguaje de Definición de Datos. Es el conjunto de comandos que se usa para crear bases de datos y tablas, y para modificar sus estructuras, así como los permisos y privilegios.
Se caracteriza por involucrar los comandos necesarios para crear, modificar y eliminar una tabla; además de crear claves primarias, índices y restricciones.

Los comandos más conocidos son:
CREATE DATABASE: Se utiliza para crear una nueva base de datos vacía.
DROP DATABASE: Se utiliza para eliminar completamente una base de datos existente.
CREATE TABLE: Crea una nueva tabla en una base de datos.
ALTER TABLE: Modifica una tabla en una base de datos existente.

DROP TABLE: Elimina una tabla en una base de datos.
CREATE INDEX: Crea un índice en una tabla o vista.
DROP INDEX: Elimina un índice de una tabla o vista.

DML

DML → Data Manipulation Language: Lenguaje de Manipulación de Datos. Es el que se usa para modificar y obtener datos desdelas bases de datos.
Se caracteriza por involucrar los comandos necesarios para hacer consultas, inserciones, modificaciones y eliminaciones.

Los comandos utilizados son:
INSERT INTO: Inserta información en una base de datos.
SELECT: Obtiene información de una base de datos.
UPDATE: Actualiza información de una base de datos.
DELETE: Elimina información de una base de datos.

DCL

Es un lenguaje que incluye una serie de comandos SQL. Como los anteriores, es proporcionado por los sistemas gestores de bases de datos. Sus siglas son **DCL** por su nombre en inglés, **Data Control Language**.

Estos comandos **permiten al Administrador del sistema gestor de base de datos, controlar el acceso a los objetos**, es decir, podemos otorgar o denegar permisos a uno o más roles para realizar determinadas tareas.

Los comandos para controlar los permisos son los siguientes:

- GRANT, permite otorgar permisos.
- REVOKE, elimina los permisos que previamente se han concedido.

TCL

TCL Transactional Control Language. Permite administrar diferentes transacciones que ocurren dentro de una **base de datos**.

- **COMMIT:** Empleado para guardar el trabajo hecho.
- **ROLLBACK:** Utilizado para deshacer la modificación que hice desde el último COMMIT.

ANSI SQL

En la actualidad SQL es el estándar *de facto* de la inmensa mayoría de los SGBD comerciales. Y, aunque la diversidad de añadidos particulares que incluyen las distintas implementaciones comerciales del lenguaje es amplia, el soporte al estándar SQL-92 es general y muy amplio.

El ANSI SQL sufrió varias revisiones y agregados a lo largo del tiempo:

Año	Nombre	Alias	Comentarios

1986	SQL-86	SQL-87	Primera publicación hecha por ANSI. Confirmada por ISO en 1987.
1989	SQL-89		Revisión menor.
1992	SQL-92	SQL2	Revisión mayor.
1999	SQL:1999	SQL2000	Se agregaron expresiones regulares, consultas recursivas (para relaciones jerárquicas), triggers y algunas características orientadas a objetos.
2003	SQL:2003		Introduce algunas características de XML, cambios en las funciones, estandarización del objeto sequence y de las columnas autonuméricas.3
2005	SQL:2005		ISO/IEC 9075-14:2005 Define las maneras en las cuales SQL se puede utilizar conjuntamente con XML. Define maneras de importar y guardar datos XML en una base de datos SQL, manipulándolos dentro de la base de datos y publicando el XML y los datos SQL convencionales en forma XML. Además, proporciona facilidades que permiten a las aplicaciones integrar dentro de su código SQL el uso de XQuery, lenguaje de consulta XML publicado por el W3C (World Wide Web Consortium) para acceso concurrente a datos ordinarios SQL y documentos XML.
2008	SQL:2008		Permite el uso de la cláusula ORDER BY fuera de las definiciones de los cursores. Incluye los disparadores del tipo INSTEAD OF. Añade la sentencia TRUNCATE.
2011	SQL:2011		Datos temporales (PERIOD FOR). Mejoras en las funciones de ventana y de la cláusula FETCH.
2016	SQL:2016		Permite búsqueda de patrones, funciones de tabla polimórficas y compatibilidad con los ficheros JSON.

Creación de una Base de Datos

El comando CREATE TABLE permite crear una BD.

```
CREATE DATABASE nombreBD;
```

Creación de una Tabla

La sentencia CREATE TABLE creará una tabla con las columnas que indiquemos.

```
CREATE TABLE Productos (
      idProducto INT (11)  UNSIGNED NOT NULL PRIMARY_KEY AUTO_INCREMENT,
      Nombre VARCHAR (50)  NOT NULL,
      Precio DOUBLE NULL ,
      Marca VARCHAR (30) NOT NULL,
      Categoria VARCHAR(30) NOT NULL,
      Presentacion VARCHAR(30) NOT NULL,
      Stock INT (6) NOT NULL,
      Disponible BOOLEAN NULL DEFAULT false
```

```
);
```

Eliminar una Tabla

```
DROP TABLE Productos;
DROP TABLE IF EXISTS Productos;
```

Nota: *Sin* la cláusula IF EXISTS devuelve error en caso de que no exista la tabla a eliminar.

Restricciones de las Tablas

Puntos claves
1- Los nombres de las tablas deben ser únicos en la base de datos.
2- Los nombres de las columnas debe ser únicos en la tabla.
3- No pueden haber dos registros con el mismo valor de la clave primaria.

Columnas No Descomponibles
1- Son aquellas columnas que contienen cierta información que no puede ser en dos o más columnas.
2- Son fáciles de actualizar.
3- Son fáciles de consultar.
4- Mejores para mantener la integridad de los datos.

Restricciones en las columnas

NOT NULL (no permite valores nulos, que es lo mismo que estar vacía).2- NO DUPLICATES (no permite nombres de columnas duplicadas)

Clave Primaria (PRIMARY KEY)
Una tabla suele tener una columna o una combinación de columnas cuyos valores identifican de forma única a cada registro de la tabla. Estas columnas se denominan claves principales de la tabla y exigen la integridad de entidad de la tabla (un solo registro con ese valor de indicador único).
Puede crear una clave principal mediante la definición de una restricción PRIMARY KEY cuando cree o modifique una tabla.
Una tabla sólo puede tener una restricción PRIMARY KEY y ninguna columna a la que se aplique una restricción PRIMARY KEY puede aceptar valores NULL. Debido a que las restricciones PRIMARY KEY garantizan datos únicos, con frecuencia se definen en una columna de identidad.
Cuando se especifica una restricción del tipo PRIMARY KEY en una tabla, el Motor de Base de Datos exige la unicidad de los datos mediante la creación de un índice único para las columnas de la clave principal. Este índice también permite un acceso rápido a los datos cuando se utiliza la clave principal en las consultas. De esta forma, las claves principales que se eligen deben seguir las reglas para crear índices únicos.
Si se define una restricción PRIMARY KEY para más de una columna, puede haber valores duplicados dentro de la misma columna, pero cada combinación de valores de todas las columnas de la definición de la restricción PRIMARY KEY debe ser única.

Otros Comandos MySQL

En las siguientes líneas se detallan comando necesarios para la navegación dentro del motor MySQL que no se encuentran contemplados en el ANSI SQL.

Comando DESCRIBE

```
DESCRIBE NombreDeTabla;
```

Devuelve la descripción de campos y detalles de una tabla (Metadato).

Comando SHOW DATABASES

```
SHOW DATABASES;
```

Muestra el catálogo de base de datos del servidor.

Comando SHOW TABLES

```
SHOW TABLES;
```

Muestra el catálogo de tablas de la base de datos.

Comando USE

```
USE testBD;
```

Activa una base de datos, para su uso. Es obligatorio tener una base de datos activa para poder operar.

Insertar registros en la tabla

Para insertar datos en una tabla utilizamos la orden INSERT. Con ella podemos ir añadiendo registros uno a uno, o añadir de golpe tantos registros como deseemos.

La sintaxis genérica de INSERT para crear un nuevo registro es la siguiente:

INSERT INTO NombreTabla (Columna1, ..., ColumnaX) VALUES (Valor1, ..., ValorX)

... siendo ...

● NombreTabla: la tabla en la que se van a insertar las filas o registros.

● (Campo1, ..., CampoX): representa el campo o campos en los que vamos a introducir valores.

● (Valor1, ..., ValorX): representan los valores que se van a almacenar en cada campo.

Ejemplo:

INSERT INTO Productos (Nombre, Precio, Marca, Categoria, Presentacion, Stock, Disponible) VALUES ('iPhone 5' , 499.99 , 'Apple' , 'Smartphone' , '16GB', 500, false);

Los campos deben coincidir exactamente igual a como han sido definidos mediante el comando CREATE.

Los valores se deben corresponder con cada uno de los campos que aparecen en la lista de campos, tanto en el tipo de dato que contienen como en el orden en el que se van a asignar. Es decir, si se indican una serie de campos en un orden determinado, la lista de valores debe especificar los valores a almacenar en dichos campos, en el mismo orden exactamente. Si un campo no está en la

lista, se almacenará dentro de éste el valor NULL.

Valores NULL

La expresión NULL significa "dato desconocido" o "valor inexistente". No es lo mismo que un valor 0, una cadena vacía o una cadena de texto literal con la palabra NULL.

A veces, puede desconocerse o no existir el dato correspondiente a algún campo de un registro. En estos casos decimos que el campo puede contener valores nulos. Por ejemplo, en la tabla de Productos se puede tener valores nulos en el campo Precio porque es posible que para algunos productos no se haya establecido el precio para la venta.

En contraposición, tenemos campos que no pueden estar vacíos jamás, por ejemplo, los campos que identifican cada registro, como los códigos de identificación, que son clave primaria.

Por defecto, es decir, si no lo aclaramos en la creación de la tabla, los campos permiten valores nulos.

Por ejemplo:

INSERT INTO Productos (Nombre, Precio, Marca, Categoria, Presentacion, Stock) VALUES ('iPhone7S', NULL, 'Apple', 'Smartphone', '16GB', 500);

Nótese que para el campo Precio el valor NULL no es una cadena de caracteres, por lo que no se coloca entre comillas. También si un campo acepta valores nulos, podemos ingresar NULL cuando no conocemos el valor.

Además, si una columna fue definida en el comando CREATE con la especificación NULL (como ser en el campo Disponible), puede no incluirse en el listado de campos (puede entenderse como que el campo "no es obligatorio" de asignar un valor).

Selección de registros

Sentencia Select

La sentencia SELECT va a permitir realizar operaciones de selección, ordenación, agrupación y filtrado de registros, es decir:

- Se utiliza para seleccionar información registrada en una tabla.
- Para seleccionar todas la columnas se utiliza el * (asterisco)
- La cláusula WHERE se utiliza para establecer un criterio de búsqueda

Sintaxis #1: Seleccionar de la tabla Productos los valores de todas las columnas de todos los registros

```
SELECT * FROM Productos;
```

Sintaxis #2: Seleccionar de la tabla Productos los valores de la columna Nombre de todos los registros

```
SELECT Nombre FROM Productos;
```

Sintaxis #3: Seleccionar de la tabla Productos todos los valores de la columna Nombre y Precio de todos los registros

```
SELECT Nombre, Precio FROM Productos;
```

Sintaxis #4: Seleccionar de la tabla Productos la columna Nombre del registro cuyo valor de la columna idProducto es igual a 1

```
SELECT Nombre FROM Productos WHERE idProducto = 1;
```

Sintaxis #5: Seleccionar de la tabla Productos las columnas Nombre y Precio los registros cuyo valor de la columna Precio sea Mayor a 150

```
SELECT Nombre, Precio FROM Productos WHERE Precio > 150;
```

Condiciones Lógicas

Para crear expresiones lógicas disponemos de varios operadores de comparación. Estos operadores se aplican a cualquier tipo de columna: fechas, cadenas, números, etc, y devuelven valores lógicos: verdadero o falso (o bien 1 ó 0).

Si uno o los dos valores a comparar son NULL , el resultado es NULL, excepto con el operador <=> que es usado para una comparación con NULL segura.

El operador <=> funciona igual que el operador =, salvo que si en la comparación una o ambas de las expresiones es nula el resultado no es NULL. Si se comparan dos expresiones nulas, el resultado es verdadero.

Tabla de Operadores Lógicos

Operador	Descripción
=	Igual
<	Menor
>	Mayor
>=	Mayor o igual
<=	Menor o igual
<>	No es igual

Los operadores IS NULL e IS NOT NULL sirven para verificar si una expresión determinada es o no nula.

Ejemplos de aplicación:

```
SELECT * FROM Productos WHERE Precio >= 500 OR Stock >= 100;
SELECT Nombre, Presentacion, Precio, Stock FROM Productos WHERE Stock < 20 AND
Precio >= 100 ;
SELECT Nombre, Presentacion FROM Productos WHERE Precio  IS NOT NULL;
```

Cláusulas Especiales

Sentencia BETWEEN

Entre los operadores de MySQL, hay uno para comprobar si una expresión está comprendida en un determinado rango de valores. La sintaxis es:

- BETWEEN mínimo AND máximo
- NOT BETWEEN mínimo AND máximo

Ejemplo de aplicación:

```
SELECT * FROM Productos WHERE Precio BETWEEN 100 AND 200;
SELECT * FROM Productos WHERE Precio NOT BETWEEN 100 AND 200;
```

Sentencia IN

Los operadores IN y NOT IN sirven para averiguar si el valor de una expresión determinada está dentro de un conjunto indicado

- IN (<expr1>, <expr2>, <expr3>,...)
- NOT IN (<expr1>, <expr2>, <expr3>,…)

El operador IN devuelve un valor verdadero si el valor de la expresión es igual a alguno de los valores especificados en la lista. El operador NOT IN devuelve un valor falso en el mismo caso. Por ejemplo:

```
SELECT 10 IN ( 2 , 4 , 6 , 8 , 10 );
```

Ejemplo de aplicación:

```
SELECT * FROM Productos WHERE Nombre IN ('iPhone','iPad' );
SELECT * FROM Productos WHERE Nombre NOT IN ('Galaxy S5', 'TV' );
```

Sentencia LIKE

El operador LIKE se usa para hacer comparaciones entre cadenas y patrones. El resultado es verdadero (1) si la cadena se ajusta al patrón, y falso (0) en caso contrario. Tanto si la cadena como el patrón son NULL, el resultado es NULL.

La sintaxis es:

```
SELECT * FROM Productos LIKE <patrón> [ESCAPE  'carácter_escape' ]
```

Carácter	Descripción
%	Coincidencia con cualquier número de caracteres, incluso ninguno.
_	Coincidencia con un único carácter.

La comparación es independiente del tipo de los caracteres, es decir, LIKE no distingue mayúsculas de minúsculas, salvo que se indique lo contrario (ver operadores de casting):

Como siempre que se usan caracteres concretos para crear patrones, se presenta la dificultad de hacer comparaciones cuando se deben buscar precisamente esos caracteres concretos. Esta dificultad se suele superar mediante secuencias de escape. Si no se especifica nada en contra, el carácter que se usa para escapar es '\'. De este modo, si queremos que nuestro patrón contenga los caracteres '%' o '_', los escaparemos de este modo: '\%' y '_':

Como en cualquier otro lenguaje, los paréntesis se pueden usar para forzar el orden de la evaluación de determinadas operaciones dentro de una expresión. Cualquier expresión entre paréntesis adquiere mayor precedencia que el resto de las operaciones en el mismo nivel de paréntesis.

Ejemplo de aplicación:

-- Listar todos los nombres que contienen TV

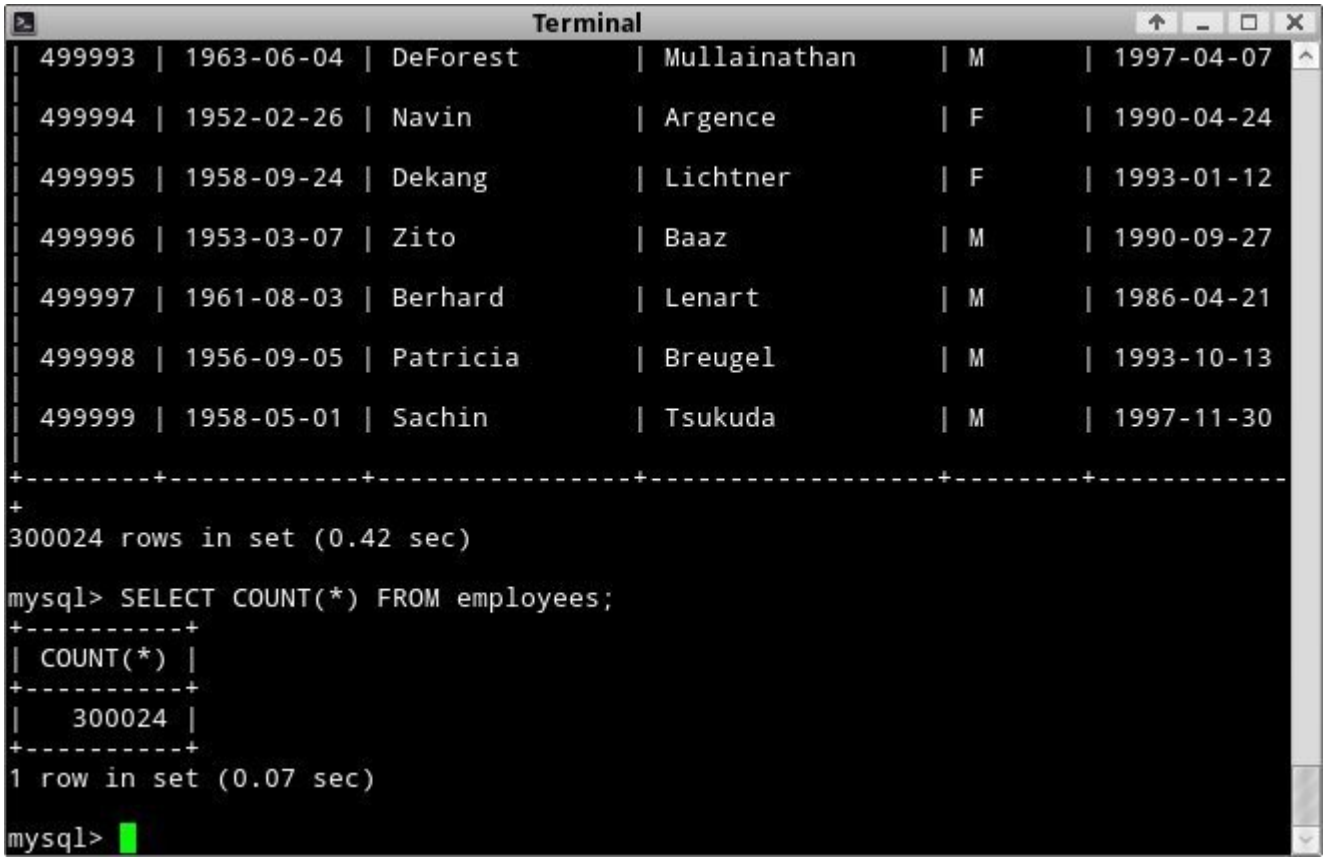
```
SELECT * FROM Productos WHERE Nombre LIKE '%TV%';
```

Otros ejemplos de select

SELECT COUNT(*) FROM <nombre_tabla>

Devuelve el total de registros en la tabla.

```
SELECT COUNT(*) FROM employees;
```



SELECT * FROM <nombre_tabla> LIMIT <limite_filas> OFFSET <número_pagina>

Lista un número de registros limitado.

```
SELECT * FROM employees LIMIT 10;
```

```
Terminal
mysql> SELECT COUNT(*) FROM employees;
+----------+
| COUNT(*) |
+----------+
|   300024 |
+----------+
1 row in set (0.07 sec)

mysql> SELECT * FROM employees LIMIT 10;
+--------+------------+------------+-----------+--------+------------+
| emp_no | birth_date | first_name | last_name | gender | hire_date  |
+--------+------------+------------+-----------+--------+------------+
|  10001 | 1953-09-02 | Georgi     | Facello   | M      | 1986-06-26 |
|  10002 | 1964-06-02 | Bezalel    | Simmel    | F      | 1985-11-21 |
|  10003 | 1959-12-03 | Parto      | Bamford   | M      | 1986-08-28 |
|  10004 | 1954-05-01 | Chirstian  | Koblick   | M      | 1986-12-01 |
|  10005 | 1955-01-21 | Kyoichi    | Maliniak  | M      | 1989-09-12 |
|  10006 | 1953-04-20 | Anneke     | Preusig   | F      | 1989-06-02 |
|  10007 | 1957-05-23 | Tzvetan    | Zielinski | F      | 1989-02-10 |
|  10008 | 1958-02-19 | Saniya     | Kalloufi  | M      | 1994-09-15 |
|  10009 | 1952-04-19 | Sumant     | Peac      | F      | 1985-02-18 |
|  10010 | 1963-06-01 | Duangkaew  | Piveteau  | F      | 1989-08-24 |
+--------+------------+------------+-----------+--------+------------+
10 rows in set (0.00 sec)

mysql>
```

También podemos definir el número de página deseado.

```
SELECT * FROM employees LIMIT 10 OFFSET 0;
```

```
Terminal
|  15332 | 1961-12-29 | Aamer      | Slutz      | F      | 1989-05-19 |
|  20678 | 1963-12-25 | Aamer      | Parveen    | F      | 1987-03-25 |
|  22279 | 1959-01-30 | Aamer      | Kornyak    | M      | 1985-02-25 |
|  23269 | 1952-02-15 | Aamer      | Szmurlo    | M      | 1988-05-25 |
|  24404 | 1960-04-21 | Aamer      | Tsukuda    | M      | 1998-12-25 |
|  28043 | 1957-07-13 | Aamer      | Kroll      | F      | 1986-05-17 |
+--------+------------+------------+------------+--------+------------+
10 rows in set (0.00 sec)

mysql> SELECT * FROM employees LIMIT 10 OFFSET 0;
+--------+------------+------------+-----------+--------+------------+
| emp_no | birth_date | first_name | last_name | gender | hire_date  |
+--------+------------+------------+-----------+--------+------------+
|  10001 | 1953-09-02 | Georgi     | Facello   | M      | 1986-06-26 |
|  10002 | 1964-06-02 | Bezalel    | Simmel    | F      | 1985-11-21 |
|  10003 | 1959-12-03 | Parto      | Bamford   | M      | 1986-08-28 |
|  10004 | 1954-05-01 | Chirstian  | Koblick   | M      | 1986-12-01 |
|  10005 | 1955-01-21 | Kyoichi    | Maliniak  | M      | 1989-09-12 |
|  10006 | 1953-04-20 | Anneke     | Preusig   | F      | 1989-06-02 |
|  10007 | 1957-05-23 | Tzvetan    | Zielinski | F      | 1989-02-10 |
|  10008 | 1958-02-19 | Saniya     | Kalloufi  | M      | 1994-09-15 |
|  10009 | 1952-04-19 | Sumant     | Peac      | F      | 1985-02-18 |
|  10010 | 1963-06-01 | Duangkaew  | Piveteau  | F      | 1989-08-24 |
+--------+------------+------------+-----------+--------+------------+
10 rows in set (0.00 sec)

mysql>
```

Para obtener la siguiente página basta con incrementar el **OFFSET**.

```
SELECT * FROM employees LIMIT 10 OFFSET 1;
```

```
Terminal
|  10005 | 1955-01-21 | Kyoichi    | Maliniak  | M      | 1989-09-12 |
|  10006 | 1953-04-20 | Anneke     | Preusig   | F      | 1989-06-02 |
|  10007 | 1957-05-23 | Tzvetan    | Zielinski | F      | 1989-02-10 |
|  10008 | 1958-02-19 | Saniya     | Kalloufi  | M      | 1994-09-15 |
|  10009 | 1952-04-19 | Sumant     | Peac      | F      | 1985-02-18 |
|  10010 | 1963-06-01 | Duangkaew  | Piveteau  | F      | 1989-08-24 |
+--------+------------+------------+-----------+--------+------------+
10 rows in set (0.00 sec)

mysql> SELECT * FROM employees LIMIT 10 OFFSET 1;
+--------+------------+------------+-----------+--------+------------+
| emp_no | birth_date | first_name | last_name | gender | hire_date  |
+--------+------------+------------+-----------+--------+------------+
|  10002 | 1964-06-02 | Bezalel    | Simmel    | F      | 1985-11-21 |
|  10003 | 1959-12-03 | Parto      | Bamford   | M      | 1986-08-28 |
|  10004 | 1954-05-01 | Chirstian  | Koblick   | M      | 1986-12-01 |
|  10005 | 1955-01-21 | Kyoichi    | Maliniak  | M      | 1989-09-12 |
|  10006 | 1953-04-20 | Anneke     | Preusig   | F      | 1989-06-02 |
|  10007 | 1957-05-23 | Tzvetan    | Zielinski | F      | 1989-02-10 |
|  10008 | 1958-02-19 | Saniya     | Kalloufi  | M      | 1994-09-15 |
|  10009 | 1952-04-19 | Sumant     | Peac      | F      | 1985-02-18 |
|  10010 | 1963-06-01 | Duangkaew  | Piveteau  | F      | 1989-08-24 |
|  10011 | 1953-11-07 | Mary       | Sluis     | F      | 1990-01-22 |
+--------+------------+------------+-----------+--------+------------+
10 rows in set (0.00 sec)

mysql>
```

Este es el concepto de una paginación.

SELECT * FROM <nombre_tabla> ORDER BY <nombre_campo>

Lista los registros ordenados por un campo. El ordenamiento puede ser ascendente o descendente.

```
SELECT * FROM employees ORDER BY first_name ASC LIMIT 10;
SELECT * FROM employees ORDER BY first_name DESC LIMIT 10;
```

Si ven, podemos combinarlos con las otras sentencias.

```
SELECT * FROM employees ORDER BY first_name ASC LIMIT 10 OFFSET 1;
```

```
Terminal
mysql> SELECT * FROM employees ORDER BY first_name ASC LIMIT 10 OFFSET 1;
+--------+------------+------------+-------------+--------+------------+
| emp_no | birth_date | first_name | last_name   | gender | hire_date  |
+--------+------------+------------+-------------+--------+------------+
|  11935 | 1963-03-23 | Aamer      | Jayawardene | M      | 1996-10-26 |
|  12160 | 1954-12-11 | Aamer      | Garrabrants | M      | 1989-09-19 |
|  13011 | 1955-02-25 | Aamer      | Glowinski   | F      | 1989-10-08 |
|  15332 | 1961-12-29 | Aamer      | Slutz       | F      | 1989-05-19 |
|  20678 | 1963-12-25 | Aamer      | Parveen     | F      | 1987-03-25 |
|  22279 | 1959-01-30 | Aamer      | Kornyak     | M      | 1985-02-25 |
|  23269 | 1952-02-15 | Aamer      | Szmurlo     | M      | 1988-05-25 |
|  24404 | 1960-04-21 | Aamer      | Tsukuda     | M      | 1998-12-25 |
|  28043 | 1957-07-13 | Aamer      | Kroll       | F      | 1986-05-17 |
|  28162 | 1960-04-06 | Aamer      | Parhami     | F      | 1996-02-15 |
+--------+------------+------------+-------------+--------+------------+
10 rows in set (0.24 sec)

mysql> SELECT MAX(hire_date) FROM employees;
+----------------+
| MAX(hire_date) |
+----------------+
| 2000-01-28     |
+----------------+
1 row in set (0.14 sec)

mysql>
```

SELECT MAX|MIN(<nombre_campo>) FROM <nombre_tabla>

Muestra el valor mayor o menor del campo en la tabla.

```
SELECT MAX(hire_date) FROM employees;
SELECT MIN(hire_date) FROM employees;
```

```
Terminal
|  15332 | 1961-12-29 | Aamer      | Slutz      | F      | 1989-05-19 |
|  20678 | 1963-12-25 | Aamer      | Parveen    | F      | 1987-03-25 |
|  22279 | 1959-01-30 | Aamer      | Kornyak    | M      | 1985-02-25 |
|  23269 | 1952-02-15 | Aamer      | Szmurlo    | M      | 1988-05-25 |
|  24404 | 1960-04-21 | Aamer      | Tsukuda    | M      | 1998-12-25 |
|  28043 | 1957-07-13 | Aamer      | Kroll      | F      | 1986-05-17 |
|  28162 | 1960-04-06 | Aamer      | Parhami    | F      | 1996-02-15 |
+--------+------------+------------+------------+--------+------------+
10 rows in set (0.24 sec)

mysql> SELECT MAX(hire_date) FROM employees;
+----------------+
| MAX(hire_date) |
+----------------+
| 2000-01-28     |
+----------------+
1 row in set (0.14 sec)

mysql> SELECT MIN(hire_date) FROM employees;
+----------------+
| MIN(hire_date) |
+----------------+
| 1985-01-01     |
+----------------+
1 row in set (0.10 sec)

mysql>
```

SELECT DISTINCT(<nombre_campo>) FROM <nombre_tabla>

Muestra los los diferentes valores de un campo en la tabla. Ningún valor aparecerá repetido.

```
SELECT DISTINCT(gender) FROM employees;
```

SELECT <nombre_campo> FROM <nombre_tabla>

Lista campos específicos de la tabla. Si hay más de un campo, estos deben delimitarse por una **”,”**.

```
SELECT first_name, last_name, gender FROM employees LIMIT 10;
```

```
Terminal
mysql> SELECT first_name, last_name, gender FROM employees LIMIT 10;
+------------+-----------+--------+
| first_name | last_name | gender |
+------------+-----------+--------+
| Georgi     | Facello   | M      |
| Bezalel    | Simmel    | F      |
| Parto      | Bamford   | M      |
| Chirstian  | Koblick   | M      |
| Kyoichi    | Maliniak  | M      |
| Anneke     | Preusig   | F      |
| Tzvetan    | Zielinski | F      |
| Saniya     | Kalloufi  | M      |
| Sumant     | Peac      | F      |
| Duangkaew  | Piveteau  | F      |
+------------+-----------+--------+
10 rows in set (0.00 sec)

mysql> SELECT SUM(salary) FROM salaries;
+--------------+
| SUM(salary)  |
+--------------+
| 181480757419 |
+--------------+
1 row in set (0.00 sec)

mysql>
```

SELECT SUM(<nombre_campo>) FROM <nombre_tabla>

Devuelve la sumatoria de campos numéricos.

```
SELECT SUM(salary) FROM salaries;
```

SELECT * FROM <nombre_tabla> WHERE <condiciones>

La sentencia **WHERE** nos sirve para filtrar registros por una serie de condiciones definidas por nosotros. Las condiciones deben cumplirse y pueden ser anidadas por los operadores lógicos **OR** y **AND**.

```
SELECT * FROM employees WHERE gender = 'M' AND last_name = 'Facello' LIMIT 10;
```

En este ejemplo estamos filtrando los registros donde el género sea **M** y además el apellido sea igual a **Facello**.

```
SELECT * FROM employees WHERE gender = 'M' AND (last_name = 'Facello' OR
last_name = 'Simmel') LIMIT 10;
```

En el segundo ejemplo el campo género debe ser **M** y el apellido puede ser **Facello** o **Simmel**. Cuando se trabaja con diferentes operadores lógicos es necesario agruparlos por paréntesis para evitar malas interpretaciones del motor SQL.

Existen otros operadores condicionales como por ejemplo:

> mayor que

```
SELECT * FROM <nombre_tabla> WHERE <nombre_campo> > <valor>;
```

< menor que

```
SELECT * FROM <nombre_tabla> WHERE <nombre_campo> < <valor>;
```

>= mayor o igual que

```
SELECT * FROM <nombre_tabla> WHERE <nombre_campo> >= <valor>;
```

<= menor o igual que

```
SELECT * FROM <nombre_tabla> WHERE <nombre_campo> <= <valor>;
```

<> diferente a

```
SELECT * FROM <nombre_tabla> WHERE <nombre_campo> <> <valor>;
```

!= no igual que

```
SELECT * FROM <nombre_tabla> WHERE <nombre_campo> != <valor>;
```

IS NULL nulo

```
SELECT * FROM <nombre_tabla> WHERE <nombre_campo> IS NULL;
```

IS NOT NULL no nulo

```
SELECT * FROM <nombre_tabla> WHERE <nombre_campo> IS NOT NULL;
```

Consultas avanzadas

Para extraer los datos y que se presenten en pantalla se utiliza la orden SELECT. Su completa constitucion seria:

```
SELECT {NombreCampo, NombreCampo, ...}
FROM NombreTablaWHERE Condiciones
[ GROUP BY { NombreCampo | Expresión | Posición } ]
[ HAVING Condiciones ]
[ ORDER BY { NombreCampo | Expresión | Posición } ]
[ ASC | DESC ]
[ LIMIT { n }, { n } ]
```

Mediante la sentencia SELECT es posible hacer una proyección de una tabla, seleccionando las columnas de las que queremos obtener datos.
Las expresiones SELECT no se limitan a nombres de columnas de tablas, pueden ser otras expresiones, incluso aunque no correspondan a ninguna tabla.

Es posible aplicar funciones sobre columnas de tablas, y usar esas columnas en expresiones para generar nuevas columnas. Por ejemplo:

```
SELECT idVenta, Fecha, DATEDIFF ( CURRENT_DATE(), Fecha ) / 30 FROM Ventas;
```

Es posible asignar un alias a cualquiera de las expresiones select. Esto se puede hacer usando la palabra AS, aunque esta palabra es opcional:

```
SELECT idVenta, Fecha, DATEDIFF(CURRENT_DATE(), Fecha ) / 30 Días FROM Ventas;
```

Comando DML Insert

Una sentencia *INSERT* de SQL agrega uno o más registros a una (y sólo una) tabla en una base de datos relacional.

Forma básica

```
INSERT INTO 'tablatura' ('columnaA',['columnaB,... '])
      VALUES ('valor1', ['valor2,...'])
```

O también se puede utilizar como:

```
INSERT INTO tablatura VALUES ('valor1','valor2')
```

Las cantidades de columnas y valores deben ser iguales. Si una columna no se especifica, le será asignado el valor por omisión. Los valores especificados (o implícitos) por la sentencia `INSERT` deberán satisfacer todas las restricciones aplicables. Si ocurre un error de sintaxis o si alguna de las restricciones es violada, no se agrega la fila y se devuelve un error.

Ejemplo

```
INSERT INTO agenda_telefonica (nombre, numero)
      VALUES ('Roberto Jeldrez',    4886850);
```

Cuando se especifican todos los valores de una tabla, se puede utilizar la sentencia acortada:

```
INSERT INTO nombreTabla VALUES ('valor1', ['valor2,...'])
```

Ejemplo (asumiendo que 'nombre' y 'número' son las únicas columnas de la tabla 'agenda_telefonica'):

```
INSERT INTO agenda_telefonica VALUES ('Jhonny Aguilar', 080473968);
```

Formas avanzadas

Una característica de SQL (desde SQL-92) es el uso de *constructores de filas* para insertar múltiples filas a la vez, con una sola sentencia SQL:

```
INSERT INTO ''tabla'' (''columna1'', ['']columna2,... '']) 
VALUES (''valor1a'', ['']valor1b,...'']),
(''value2a'', ['']value2b,...'']),...;
```

Esta característica es soportada por DB2, PostgreSQL (desde la versión 8.2), MySQL, y H2.

Ejemplo (asumiendo que 'nombre' y 'número' son las únicas columnas en la tabla 'agenda_telefonica'):

```
INSERT INTO agenda_telefonica VALUES ('Roberto Fernández', '4886850'),
('Alejandro Sosa', '4556550');
```

Que podía haber sido realizado por las sentencias

```
INSERT INTO agenda_telefonica VALUES ('Roberto Fernández', '4886850');
INSERT INTO agenda_telefonica VALUES ('Alejandro Sosa', '4556550');
```

Notar que las sentencias separadas pueden tener semántica diferente (especialmente con respecto a los triggers), y puede tener diferente rendimiento que la sentencia de inserción múltiple.

Para insertar varias filas en MS SQL puede utilizar esa construcción:

```
INSERT INTO phone_book SELECT 'John Doe', '555-1212'
UNION ALL SELECT 'Peter Doe', '555-2323';
```

Tenga en cuenta que no se trata de una sentencia SQL válida de acuerdo con el estándar SQL (SQL: 2003), debido a la cláusula subselect incompleta.

Para hacer lo mismo en Oracle se usa la Tabla DUAL, siempre que se trate de solo una simple fila:

```
INSERT INTO phone_book
SELECT 'John Doe', '555-1212' FROM DUAL
UNION ALL
SELECT 'Peter Doe','555-2323' FROM DUAL
```

Una implementación conforme al estándar de esta lógica se muestra el siguiente ejemplo, o como se muestra arriba (no aplica en Oracle):

```
INSERT INTO phone_book
SELECT 'John Doe', '555-1212' FROM LATERAL ( VALUES (1) ) AS t(c)
UNION ALL
SELECT 'Peter Doe','555-2323' FROM LATERAL ( VALUES (1) ) AS t(c)
```

Copia de filas de otras tablas

Un INSERT también puede utilizarse para recuperar datos de otros, modificarla si es necesario e insertarla directamente en la tabla. Todo esto se hace en una sola sentencia SQL que no implica ningún procesamiento intermedio en la aplicación cliente. Un SUBSELECT se utiliza en lugar de la cláusula VALUES. El SUBSELECT puede contener JOIN, llamadas a funciones, y puede incluso consultar en la misma TABLA los datos que se inserta. Lógicamente, el SELECT se evalúa antes que la operación INSERT esté iniciada. Un ejemplo se da a continuación.

```
INSERT INTO phone_book2
SELECT * FROM phone_book WHERE name IN ('John Doe', 'Peter Doe');
```

Una variación es necesaria cuando algunos de los datos de la tabla fuente se está insertando en la nueva tabla, pero no todo el registro. (O cuando los esquemas de las tablas no son iguales.)

```
INSERT INTO phone_book2 ( [name], [phoneNumber] )
SELECT [name], [phoneNumber] FROM phone_book
WHERE name IN ('John Doe', 'Peter Doe');
```

El SELECT produce una tabla (temporal), y el esquema de la tabla temporal debe coincidir con el esquema de la tabla donde los datos son insertados.

Comando DML Delete

Para eliminar filas se usa la sentencia DELETE. La sintaxis genérica de DELETE para eliminar un registro existente es la siguiente:

```
DELETE FROM NombreTabla WHERE NombreCampo Operador Valor];
```

Los filtros que podemos utilizar dentro de la cláusula WHERE son los mismos que se pueden ver en las instrucciones SELECT ó bien UPDATE.

Ejemplo de aplicación:

```
DELETE FROM Productos WHERE idProducto = 1;
```

Si no aplicamos ningún filtro se eliminarán todos los registros sin ninguna limitación.

Si se borran mucho registros y se usa el cliente MySQL workbench es necesario quitar la protección de borrado masivo.
Ejemplo de como desactivar safe updates:

```
Set sql_safe_updates=0;
```

La instrucción DELETE consume muchos recursos si se la usa para eliminar todos los registros, por este motivo si se opta por vaciar completamente la tabla es recomendable utilizar la instrucción TRUNCATE TABLE, la cual elimina los registros en su totalidad dejando vacía la tabla y de manera menos traumática para el servidor de base de datos, con la siguiente sintaxis:

TRUNCATE TABLE NombreTabla;

Comando DML Update

Podemos modificar valores de las filas de una tabla usando la sentencia UPDATE. La sintaxis genérica de UPDATE para actualizar un registro existente es la siguiente:

```
UPDATE NombreTabla SET NombreCampo = "Valor1" [, NombreCampo2 = "Valor2"]
[ WHERE NombreCampo Operador Valor]
```

Es posible del mismo modo, actualizar el valor de más de una columna separándolas en la sección SET mediante comas como delimitador.

Actualizaciones selectivas

Mediante la cláusula WHERE se puede establecer una condición y sólo las filas/registros que cumplan esa condición serán
actualizadas.
Ejemplo de aplicación:

```
UPDATE Productos SET Nombre = 'iPhone 6' WHERE Nombre = 'iPhone Seis';
UPDATE Productos SET Precio = '499.99' WHERE idProducto = 10;
UPDATE Productos SET Stock = 600 , Precio = '499.99' WHERE idProducto = 15;
```

NOTA: Si el resultado es (0 row(s) affected) quiere decir que no hay registros que tengan el valor de la columna indicada en el WHERE

Funciones de Agrupamiento

Existen en SQL funciones que nos permiten contar registros, calcular sumas, promedios, obtener valores máximos y mínimos. Estas funciones se denominan Funciones de Agrupamiento porque operan sobre conjuntos de registros, no con datos individuales.
Tienen la característica de agrupar los resultados en un solo registro de salida.

Función COUNT()

Retorna la cantidad de valores que contiene un campo especificado. Por ejemplo, si se quiere saber la cantidad de productos que hay en la tabla Productos:

```
SELECT COUNT(idProductos) FROM Productos;
```

También es posible combinarla con la cláusula WHERE. Por ejemplo si se quiere saber cuántos productos tienen por Nombre "iPhone" de en tabla Productos:

```
SELECT COUNT(idProductos) FROM Productos WHERE Nombre LIKE "%iPhone%";
```

Función SUM()

Retorna la suma de los valores que contiene el campo especificado. Por ejemplo, si se quiere saber el stock de productos que hay en la tabla Productos:

```
SELECT SUM(Stock) FROM Productos;
```

También es posible combinarla con la cláusula WHERE. Por ejemplo si se quiere saber el stock de productos "iPad" que hay para vender:

```
SELECT SUM(Stock) FROM Productos WHERE Nombre LIKE "%iPad%";
```

Funciones MIN() y MAX()

Para averiguar el valor máximo o mínimo de un campo usamos las funciones max() y min() respectivamente. Ejemplo, se quiere saber cuál es el mayor precio de todos los productos:

```
SELECT MAX(Precio) FROM Productos;
```

También si se quiere saber cuál es el valor mínimo de los productos "TV":

```
SELECT MIN(Precio) FROM Productos WHERE Nombre LIKE "%TV%";
```

Función AVG()

Retorna el valor promedio de los valores del campo especificado. Por ejemplo, si se quiere saber el promedio del precio de los productos "Impresora":

SELECT AVG(Precio) FROM Productos WHERE Nombre LIKE '%Impresora%';

Tener en cuenta que no debe haber espacio entre el nombre de la función y el paréntesis pues puede confundirse con una referencia a una tabla o campo. Es decir…

Correcto :)
SELECT COUNT(*) FROM Productos;

Incorrecto :(
SELECT COUNT (*) FROM Productos;

Agrupación con GROUP BY

La agrupación es un concepto básico de Base de Datos, La cláusula GROUP BY, como su traducción lo indica tiene como propósito agrupar información de acuerdo a un criterio en común. Por lo general se utiliza con funciones de agrupación o de agregación (COUNT, MIN, MAX, AVG, SUM). Por ejemplo usando la función SUM…

La función GROUP BY permite hacer esto de manera automática a partir de tomar un valor o dato común. Por ejemplo:

```
SELECT Categoria, SUM(Stock) FROM Productos GROUP BY Categoria;
```

El comportamiento de la función GROUP BY dependerá de la función de agrupación que se esté utilizando.

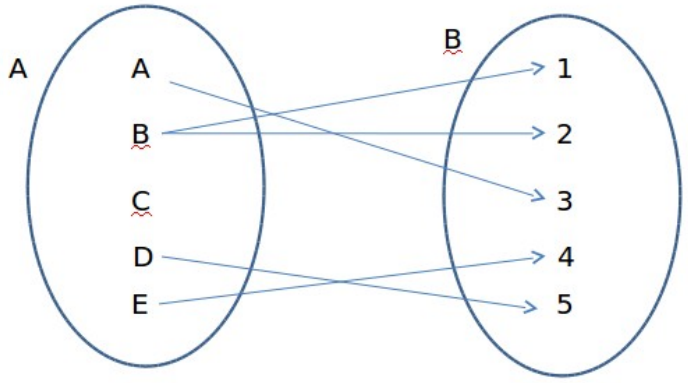
Agrupación con HAVING

La cláusula HAVING permite hacer selecciones en situaciones en las que no es posible usar WHERE. Dado que se establece un criterio sobre un valor dado por una función de agrupamiento y no por valores de registros.
Un ejemplo completo sería:

```
SELECT Categoria, SUM(Stock) FROM Productos GROUP BY Categoria HAVING SUM(Stock)
> 250;
```

Consultas de más de una tabla

Es posible hacer consultas usando varias tablas en la misma sentencia SELECT.
Esto permite realizar otras dos operaciones de álgebra relacional: el producto cartesiano y la composición interna (Producto Relacionado).
El producto cartesiano se obtiene mencionando las dos tablas en una consulta sin ninguna restricción en la cláusula WHERE.
El producto cartesiano de dos tablas son todas las combinaciones de todas las filas de las dos tablas.
Usando una sentencia SELECT se deben proyectan todos los atributos de ambas tablas. Los nombres de las tablas se indican en la cláusula FROM separados con comas.



Producto Cartesiano
A1 A2 A3 A4 A5
B1 B2 B3 B4 B5
C1 C2 C3 C4 C5
D1 D2 D3 D4 D5
E1 E2 E3 E4 E5

Producto Relacionado
A3 B1 B2 D5 E4

Ejemplo de producto cartesiano:

```
SELECT * FROM Productos, Marcas;
```

En este caso, la visualización de registros resultantes estará compuesta por el producto entre la totalidad de registros de una tabla y la totalidad de registros de la otra tabla.

Otro ejemplo puede comprobar la cantidad de registros de la misma consulta mediante la función count():

```
SELECT COUNT(*) FROM Productos, Marcas;
```

La composición interna se trata de un producto cartesiano restringido en donde las tuplas (conjunto de nombres de atributos relacionados) que se emparejan deben cumplir una determinada condición.
Ejemplo de composición interna:

```
SELECT * FROM Productos, Marcas WHERE Productos.Marca = Marcas.idMarca;
```

Uso del Join

ANSI SQL brinda una instrucción para apoyar la unión entre tablas.

```
-- consultamos usando el comando join
select * from clientes c join facturas f on c.codigo=f.codigocliente;

-- muestra todos los elementos libres de la derecha
select * from clientes c left join facturas f on c.codigo=f.codigocliente;

-- muestra todos los elementos libres de la izquierda
select * from clientes c right join facturas f on c.codigo=f.codigocliente;
```

Modelo de Entidad - Relación

Introducción

Cuando se utiliza una base de datos para gestionar información se está plasmando una parte del mundo real en una serie de tablas, registros y campos ubicados en un ordenador; creándose un modelo parcial de la realidad. Antes de crear físicamente estas tablas se debe realizar un modelo de datos.
El modelo entidad-relación (E-R) es uno de los varios modelos conceptuales existentes para el diseño de bases de datos.
Se suele cometer el error de ir creando nuevas tablas a medida que se van necesitando, haciendo así el modelo de datos y la construcción física de las tablas simultáneamente.
El modelo de datos más extendido es el denominado ENTIDAD/RELACIÓN (E/R) En el modelo E/R se parte de una situación real a partir de la cual se definen entidades y relaciones entre dichas entidades.

Entidad

Una entidad es cualquier "objeto" discreto sobre el que se tiene información. Cada ejemplar de una entidad se denomina instancia.
Las entidades son modeladas en la base de datos como tablas.

Clave Foránea (FOREIGN KEY)

La Clave Foránea referencia a la clave primaria de una tabla. Esta puede referenciar a la clave primaria de la misma tabla o de otra.
Ante una consulta SQL se valida la legitimidad de los datos almacenados en una clave foránea y se fuerza la integridad referencial.
Sintaxis del FOREIGN KEY:

```
ALTER TABLE NombreTabla ADD FOREIGN KEY (Campo1) REFERENCES TablaCategorizante
(idTabla) ON DELETE [CASCADE NO ACTION RESTRICT] ON UPDATE [CASCADE NO ACTION
RESTRICT]
```

Ejemplo:
1) Crear la tabla Productos a sabiendas que hay/existirá una tabla Marcas en donde estarán registradas varios registros que representarán las marcas existentes.
Entonces, la tabla Productos tendrá un campo "categorizante" (en este caso, Marca) al cual se le asignarán valores que coincidan con los IDs existentes de la tabla Marcas.

```
CREATE TABLE Productos (
      idProducto INT UNSIGNED NOT NULL AUTO_INCREMENT,
      Nombre VARCHAR(50) NOT NULL,
      Precio DOUBLE NULL,
      Marca INT UNSIGNED NOT NULL   -- Coincide en tipo/longitud con el campo al
                                    -- que será relacionado
      Categoria VARCHAR(30) NOT NULL,
      Presentacion VARCHAR(30) NOT NULL,
      Stock INT NOT NULL,
      Disponible BOOLEAN NULL DEFAULT false,
      PRIMARY KEY(idProducto),
      KEY(Marca) -- se defina como "campo clave" para poder asignarle una
                 -- Foreign Key
);

CREATE TABLE Marcas(
      idMarca INT UNSIGNED NOT NULL AUTO_INCREMENT,
      Nombre VARCHAR(30) NOT NULL,
      PRIMARY KEY(idMarca)
      -- Al definir una Primary Key esta podrá tener como enlace una Foreign Key
);
```

La relación se podrá entender de la siguiente forma: La Tabla Productos, en su campo Marca, se "alimenta" de la Tabla Marcas a traves de su campo idMarca.

```
ALTER TABLE Productos ADD FOREIGN KEY(Marca) REFERENCES Marcas(idMarca) ON
DELETE CASCADE ON UPDATE CASCADE;
```

Conceptos Claves

Super llave

Es un conjunto de uno o más atributo que "juntos" identifican de manera única a una entidad.
Es un conjunto de uno o más atributos que, tomados colectivamente, permiten identificar de forma única un registro en el conjunto de registros. Es un conjunto de atributos mediante los cuales es posible reconocer a un registro. Este tipo de llaves contiene comúnmente atributos ajenos; es decir, atributos que no son indispensables para llevar a cabo el reconocimiento del registro.

Clave candidata

Llave candidata: es una súper llave mínima. Una tabla puede tener varias llaves candidatas, pero solo una es elegida como llave primaria.

Relación

Una relación describe cierta interdependencia (de cualquier tipo) entre una o más entidades. Esta no tiene sentido sin las entidades que relaciona. Las relaciones son definidas con claves primarias y claves foráneas y mantienen la integridad referencial.

Cardinalidad de las Relaciones

Una relación describe cierta interdependencia (de cualquier tipo) entre una o más entidades. Las relaciones pueden ser:

- Relaciones de uno a uno: una instancia de la entidad A se relaciona con una y solamente una de la entidad B.
- Relaciones de uno a muchos: cada instancia de la entidad A se relaciona con varias instancias de la entidad B.
- Relaciones de muchos a muchos: cualquier instancia de la entidad A se relaciona con cualquier instancia de la entidad B.

Atributos

Las entidades tienen atributos. Un atributo de una entidad representa alguna propiedad que nos interesa almacenar en el modelo de Bases de Datos, los atributos son almacenados como columnas o campos de una tabla.

Consideraciones en el Planeamiento del Diseño Lógico de la Base de Datos
● Determinar el negocio y las necesidades del usuario.
● Considerando cuales son los problemas que hay que salvar y las tareas que los usuarios deberán completar.
● Crear Bases de Datos Normalizadas.
● Prever innecesariamente información duplicada, inconsistencias en la base de datos, anomalías y problemas de pérdida de la información.

Normalización de Base de Datos

La **normalización de bases de datos** es un proceso que consiste en designar y aplicar una serie de reglas a las relaciones obtenidas tras el paso del modelo entidad-relación al modelo relacional.

Las bases de datos relacionales se normalizan para:

- Evitar la redundancia de los datos.
- Disminuir problemas de actualización de los datos en las tablas.
- Proteger la integridad de datos.

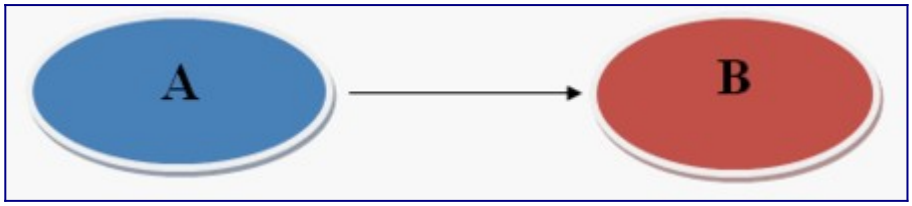
En el modelo relacional es frecuente llamar *tabla* a una relación; para que una tabla sea considerada como una relación tiene que cumplir con algunas restricciones:

- Cada tabla debe tener su nombre único.
- No puede haber dos filas iguales. No se permiten los duplicados.

- Todos los datos en una columna deben ser del mismo tipo.

Dependencias

Dependencia funcional



B es funcionalmente dependiente de **A**.

Una dependencia funcional es una conexión entre uno o más atributos. Por ejemplo, si se conoce el valor de *DNI(Documento Nacional de Identidad-España)* tiene una conexión con *Apellido* o *Nombre* .

Las dependencias funcionales del sistema se escriben utilizando una flecha, de la siguiente manera:

FechaDeNacimiento . Edad

De la normalización (lógica) a la implementación (física o real) puede ser sugerible tener estas dependencias funcionales para lograr la eficiencia en las tablas.

Propiedades de la dependencia funcional

Existen tres axiomas de Armstrong:

Dependencia funcional reflexiva

Si "y" está incluido en "x" entonces x . y

A partir de cualquier atributo o conjunto de atributos siempre puede deducirse él mismo. Si la dirección o el nombre de una persona están incluidos en el DNI, entonces con el DNI podemos determinar la dirección o su nombre.

Dependencia funcional argumentativa

. entonces .

DNI . nombre

DNI,dirección . nombre,dirección

Si con el DNI se determina el nombre de una persona, entonces con el DNI más la dirección también se determina el nombre y su dirección.

Dependencia funcional transitiva

Dependencia funcional transitiva.

Sean *X, Y, Z* tres atributos (o grupos de atributos) de la misma entidad. Si *Y* depende funcionalmente de *X* y *Z* de *Y*, pero *X* no depende funcionalmente de *Y*, se dice entonces que *Z* depende transitivamente de *X*. Simbólicamente sería:

X . *Y* . *Z* entonces *X* . *Z*

FechaDeNacimiento . *Edad*

Edad . *Conducir*

FechaDeNacimiento . *Edad* . *Conducir*

Entonces entendemos que *FechaDeNacimiento* determina a *Edad* y la *Edad* determina a *Conducir*, indirectamente podemos saber a través de *FechaDeNacimiento* a *Conducir* (En muchos países, una persona necesita ser mayor de cierta edad para poder conducir un automóvil, por eso se utiliza este ejemplo).

"C será un dato simple (dato no primario), B, será un otro dato simple (dato no primario), A, es la llave primaria (PK). Decimos que C dependerá de B y B dependerá funcionalmente de A."

Propiedades deducidas

Unión

. y . entonces .

Pseudo-Transitiva

. y . entonces .

Descomposición

. y . está incluido en . entonces .

Formas normales

Las formas normales son aplicadas a las tablas de una base de datos. Decir que una base de datos está en la forma normal **N** es decir que todas sus tablas están en la forma normal **N**.

Diagrama de inclusión de todas las formas normales.

En general, las primeras tres formas normales son suficientes para cubrir las necesidades de la mayoría de las bases de datos. El creador de estas 3 primeras formas normales (o reglas) fue Edgar F. Codd.1

Primera Forma Normal (1FN)

Artículo principal: *Primera forma normal*

Una tabla está en primera forma si.

- Todos los atributos son atómicos. Un atributo es atómico si los elementos del dominio son simples e indivisibles.
- No debe existir variación en el número de columnas.
- Los campos no clave deben identificarse por la clave (dependencia funcional).
- Debe existir una independencia del orden tanto de las filas como de las columnas; es decir, si los datos cambian de orden no deben cambiar sus significados.

Esta forma normal elimina los valores repetidos dentro de una base de datos.

Segunda Forma Normal (2FN)

Artículo principal: *Segunda forma normal*

Dependencia funcional. Una relación está en 2FN si está en 1FN y si los atributos que no forman parte de ninguna clave dependen de forma completa de la clave principal. Es decir, que no existen dependencias parciales. Todos los atributos que no son clave principal deben depender únicamente de la clave principal.

En otras palabras, podríamos decir que la segunda forma normal está basada en el concepto de dependencia completamente funcional. Una dependencia funcional . es completamente funcional si al eliminar los atributos A de X significa que la dependencia no es mantenida, esto es que .. Una dependencia funcional . es una dependencia parcial si hay algunos atributos . que pueden ser eliminados de X y la dependencia todavía se mantiene, esto es ..

Por ejemplo {DNI, ID_PROYECTO} . HORAS_TRABAJO (con el DNI de un empleado y el ID de un proyecto sabemos cuántas horas de trabajo por semana trabaja un empleado en dicho proyecto) es completamente funcional dado que ni DNI . HORAS_TRABAJO ni ID_PROYECTO .

HORAS_TRABAJO mantienen la dependencia. Sin embargo {DNI, ID_PROYECTO} . NOMBRE_EMPLEADO es parcialmente dependiente dado que DNI . NOMBRE_EMPLEADO mantiene la dependencia.

Tercera Forma Normal (3FN)

Artículo principal: *Tercera forma normal*

La tabla se encuentra en 3FN si es 2FN y si no existe ninguna dependencia funcional transitiva en los atributos que no son clave.

Un ejemplo de este concepto sería que, una dependencia funcional X . Y en un esquema de relación R es una dependencia transitiva si hay un conjunto de atributos Z que no es un subconjunto de alguna clave de R, donde se mantiene X . Z y Z . Y.

Por ejemplo, la dependencia SSN . DMGRSSN es una dependencia transitiva en EMP_DEPT de la siguiente figura. Decimos que la dependencia de DMGRSSN el atributo clave SSN es transitiva vía DNUMBER porque las dependencias SSN → DNUMBER y DNUMBER → DMGRSSN son mantenidas, y DNUMBER no es un subconjunto de la clave de EMP_DEPT. Intuitivamente, podemos ver que la dependencia de DMGRSSN sobre DNUMBER es indeseable en EMP_DEPT dado que DNUMBER no es una clave de EMP_DEPT.

Formalmente, un esquema de relación . está en 3 Forma Normal Elmasri-Navâthe,2 si para toda dependencia funcional „ se cumple al menos una de las siguientes condiciones:

1. . es superllave o clave.
2. . es atributo primo de .; esto es, si es miembro de alguna clave en „

Además el esquema debe cumplir necesariamente, con las condiciones de segunda forma normal.

Forma normal de Boyce-Codd (FNBC)

Artículo principal: *Forma normal de Boyce-Codd*

La tabla se encuentra en FNBC si cada determinante, atributo que determina completamente a otro, es clave candidata. Deberá registrarse de forma anillada ante la presencia de un intervalo seguido de una formalización perpetua, es decir las variantes creadas, en una tabla no se llegaran a mostrar, si las ya planificadas, dejan de existir.

Formalmente, un esquema de relación . está en FNBC, si y sólo si, para toda dependencia funcional . válida en „ se cumple que

1. . es superllave o clave.

De esta forma, todo esquema . que cumple FNBC, está además en 3FN; sin embargo, no todo esquema . que cumple con 3FN, está en FNBC.

Cuarta Forma Normal (4FN)

Artículo principal: *Cuarta forma normal*

Una tabla se encuentra en 4FN si, y solo si, para cada una de sus dependencias multivaluadas no funcionales X . Y, siendo X una super-clave que, X es una clave candidata o un conjunto de claves primarias.

Quinta Forma Normal (5FN)

Artículo principal: *Quinta forma normal*

Una tabla se encuentra en 5FN si:

- La tabla está en 4FN
- No existen relaciones de dependencias de reunión (join) no triviales que no se generen desde las claves. Una tabla que se encuentra en la 4FN se dice que está en la 5FN si, y sólo si, cada relación de dependencia de reunión (join) se encuentra definida por claves candidatas. Por lo que si se aplicara una consulta entre al menos tres relaciones independientes entre sí dentro de la 4FN y se obtuvieran tuplas espurias, entonces no estaría dentro de la 5FN.

Reglas de Codd

Edgar Frank Codd se percató de que existían bases de datos en el mercado que decían ser relacionales, pero lo único que hacían era guardar la información en las tablas, sin estar literalmente normalizadas dichas tablas; entonces Codd publicó doce (12) reglas que un verdadero sistema relacional debería tener, en la práctica algunas de ellas son difíciles de realizar. Un sistema podrá considerarse "más relacional" cuanto más siga estas reglas.

Regla 1: La regla de la información

Toda la información en un RDBMS está explícitamente representada de una sola manera por valores en una tabla.

Cualquier cosa que no exista en una tabla no existe del todo. Toda la información, incluyendo nombres de tablas, nombres de vistas, nombres de columnas, y los datos de las columnas deben estar almacenados en tablas dentro de las bases de datos. Las tablas que contienen tal información constituyen el Diccionario de Datos. Esto significa que todo tiene que estar almacenado en las tablas.

Toda la información en una base de datos relacional se representa explícitamente en el nivel lógico exactamente de una manera: con valores en tablas. Por tanto los metadatos (diccionario, catálogo) se representan exactamente igual que los datos de usuario. Y puede usarse el mismo lenguaje (ej. SQL) para acceder a los datos y a los metadatos (regla 4).

Regla 2: La regla del acceso garantizado

Cada ítem de datos debe ser lógicamente accesible al ejecutar una búsqueda que combine el nombre de la tabla, su clave primaria y el nombre de la columna.

Esto significa que dado un nombre de tabla, dado el valor de la clave primaria y dado el nombre de la columna requerida, deberá encontrarse uno y solamente un valor. Por esta razón la definición de claves primarias para todas las tablas es prácticamente obligatoria.

Regla 3: Tratamiento sistemático de los valores nulos

La información inaplicable o faltante puede ser representada a través de valores nulos

Un RDBMS (Sistema Gestor de Bases de Datos Relacionales) debe ser capaz de soportar el uso de valores nulos en el lugar de columnas cuyos valores sean desconocidos.

- Se reconoce la necesidad de la existencia del valor nulo, el cual podría servir para representar, o bien una información desconocida (ejemplo, no se sabe la dirección de un empleado), o bien una información que no procede (a un empleado soltero no se le puede asignar un nombre de esposa). Así mismo, consideremos el caso de un alumno que obtiene 0 puntos en una prueba y el de un alumno que no presentó la prueba.
- Hay problemas para soportar los valores nulos en las operaciones relacionales, especialmente en las operaciones lógicas, para lo cual se considera una lógica trivaluada, con tres (no dos) valores de verdad: verdadero, falso y null. Se crean tablas de verdad para las operaciones lógicas:

```
null AND null = null

Verdadero AND null = null

Falso AND null = Falso

Verdadero OR null = Verdadero, etc.
```

Regla 4: La regla de la descripción de la base de datos

La descripción de la base de datos es almacenada de la misma manera que los datos ordinarios, esto es, en tablas y columnas, y debe ser accesible a los usuarios autorizados.

La información de tablas, vistas, permisos de acceso de usuarios autorizados, etc, debe ser almacenada exactamente de la misma manera: En tablas. Estas tablas deben ser accesibles igual que todas las tablas, a través de sentencias de SQL (o similar).

Regla 5: La regla del sub-lenguaje integral

Debe haber al menos un lenguaje que sea integral para soportar la definición de datos, manipulación de datos, definición de vistas, restricciones de integridad, y control de autorizaciones y transacciones.

Esto significa que debe haber por lo menos un lenguaje con una sintaxis bien definida que pueda ser usado para administrar completamente la base de datos.

Regla 6: La regla de la actualización de vistas

Todas las vistas que son teóricamente actualizables, deben ser actualizables por el sistema mismo.

La mayoría de las RDBMS permiten actualizar vistas simples, pero deshabilitan los intentos de actualizar vistas complejas.

Regla 7: La regla de insertar y actualizar

La capacidad de manejar una base de datos con operandos simples se aplica no sólo para la recuperación o consulta de datos, sino también para la inserción, actualización y borrado de datos'.

Esto significa que las cláusulas para leer, escribir, eliminar y agregar registros (SELECT, UPDATE, DELETE e INSERT en SQL) deben estar disponibles y operables, independientemente del tipo de relaciones y restricciones que haya entre las tablas o no.

Regla 8: La regla de independencia física

El acceso de usuarios a la base de datos a través de terminales o programas de aplicación, debe permanecer consistente lógicamente cuando quiera que haya cambios en los datos almacenados, o sean cambiados los métodos de acceso a los datos.

El comportamiento de los programas de aplicación y de la actividad de usuarios vía terminales debería ser predecible basados en la definición lógica de la base de datos, y éste comportamiento debería permanecer inalterado, independientemente de los cambios en la definición física de ésta.

Regla 9: La regla de independencia lógica

Los programas de aplicación y las actividades de acceso por terminal deben permanecer lógicamente inalteradas cuando quiera que se hagan cambios (según los permisos asignados) en las tablas de la base de datos.

La independencia lógica de los datos especifica que los programas de aplicación y las actividades de terminal deben ser independientes de la estructura lógica, por lo tanto los cambios en la estructura lógica no deben alterar o modificar estos programas de aplicación.

Regla 10: La regla de la independencia de la integridad

Todas las restricciones de integridad deben ser definibles en los datos, y almacenables en el catálogo, no en el programa de aplicación.

Las reglas de integridad

1. Ningún componente de una clave primaria puede tener valores en blanco o nulos (ésta es la norma básica de integridad).
2. Para cada valor de clave foránea deberá existir un valor de clave primaria concordante. La combinación de estas reglas aseguran que haya integridad referencial.

Regla 11: La regla de la distribución

El sistema debe poseer un lenguaje de datos que pueda soportar que la base de datos esté distribuida físicamente en distintos lugares sin que esto afecte o altere a los programas de aplicación.

El soporte para bases de datos distribuidas significa que una colección arbitraria de relaciones, bases de datos corriendo en una mezcla de distintas máquinas y distintos sistemas operativos y que esté conectada por una variedad de redes, pueda funcionar como si estuviera disponible como en una única base de datos en una sola máquina.

Regla 12: Regla de la no-subversión

Si el sistema tiene lenguajes de bajo nivel, estos lenguajes de ninguna manera pueden ser usados para violar la integridad de las reglas y restricciones expresadas en un lenguaje de alto nivel (como SQL).

Algunos productos solamente construyen una interfaz relacional para sus bases de datos No relacionales, lo que hace posible la subversión (violación) de las restricciones de integridad. Esto no debe ser permitido.